Automated Smart Waste Sorting System

1st Zekarias Jemberu
Department of Electrical and Computer Engineering
Abu Dhabi University
Abu Dhabi, United Arab Emirates
Email: 1093900@students.adu.ac.ae

2nd Abdulrahman Saleh Al-Ameri
Department of Electrical and Computer Engineering
Abu Dhabi University
Abu Dhabi, United Arab Emirates
Email: 1089525@students.adu.ac.ae

3rd Michael Tsiwya
Department of Electrical and Computer Engineering
Abu Dhabi University
Abu Dhabi, United Arab Emirates
Email: 1093909@students.adu.ac.ae

***Abstract*—This report presents an IoT-enabled automated smart waste sorting system that enhances waste segregation efficiency using classical computer vision and embedded control techniques. The system employs a Raspberry Pi 4 as the main processing unit to capture images and classify waste items using feature-based object recognition with ORB descriptors and brute-force matching implemented in OpenCV. A reference database of 665 recyclable and 647 non-recyclable images supports the classification process. An Arduino Uno handles edge-level tasks, including servo motor control and ultrasonic-based bin fill-level monitoring. Based on the classification result, a servo-driven mechanical platform directs each item into the appropriate recyclable or non-recyclable bin. Experimental evaluation demonstrates a classification accuracy of approximately 92% with an average sorting time of 1.03 seconds per item, confirming stable real-time performance under controlled conditions. Fill-level measurements are transmitted via USB serial communication to the Raspberry Pi and visualized on a locally hosted Flask-based monitoring dashboard. The separation of computational processing and hardware control ensures efficient resource utilization and low-latency operation. The proposed system offers a low-cost and practical solution suitable for educational environments and small-scale smart waste management applications, with potential for future expansion to cloud connectivity and advanced recognition techniques.**

***Index Terms*—Internet of Things, Smart Waste Management, Computer Vision, Edge Computing, Environmental Sustainability, Raspberry Pi, OpenCV**

I. Introduction

The global waste management crisis represents one of the most pressing environmental challenges of our time, as traditional systems struggle to keep pace with increasing waste volumes and material complexity [1]. Manual sorting operations achieve low classification accuracy while consuming a significant portion of total recycling operational costs. This inefficiency leads to economic losses and contributes to environmental degradation through improper waste handling and reduced recycling rates.

Recent advances in Internet of Things (IoT) technologies and embedded computer vision create new opportunities to address these challenges through intelligent automation [2]. Motivated by the objectives of the CEN425 Internet of Things course, this project develops an efficient, accessible, and practical waste sorting system that bridges the gap between complex industrial solutions and deployable academic prototypes. Prior research demonstrates that computer vision techniques and distributed sensor networks improve waste classification accuracy and system responsiveness [3]. These developments inform the design philosophy of the proposed system.

The project requirements specify an IoT-enabled waste bin that automatically classifies waste items. Several critical design constraints were thoroughly addressed during implementation. Hardware constraints necessitated a hybrid architecture where the Raspberry Pi handles intensive vision tasks while the Arduino manages time-sensitive servo control. Economic constraints limited the design to low-cost, off-the-shelf components like the SG90 servo and HC-SR04 sensors to ensure affordability for educational use. Finally, environmental constraints, specifically variable indoor lighting, were mitigated through adaptive thresholding in the software pipeline to maintain classification stability. An overview of the complete system architecture and data flow appears in Fig. 1.

The proposed system integrates hardware and software components within a balanced edge-gateway architecture. The Raspberry Pi performs image acquisition and waste classification using ORB-based feature matching implemented in OpenCV, while an Arduino Uno edge node manages sensor interfacing and servo motor control. Classification accuracy is experimentally evaluated by comparing predicted waste categories against a labelled reference dataset consisting of 1312 images (665 recyclable and 647 non-recyclable). Using this method, the system achieves a measured accuracy of approximately 92%, confirming reliable real-time performance under controlled testing. A mechanical sorting mechanism complements the classification pipeline. This distribution of computational and control tasks ensures low latency, efficient resource utilization, and stable real-time performance.

The primary contributions of this project include demonstrating a practical ORB-based feature matching waste classification system on resource-constrained hardware [3], validating a hybrid edge-gateway IoT architecture for real-time operation

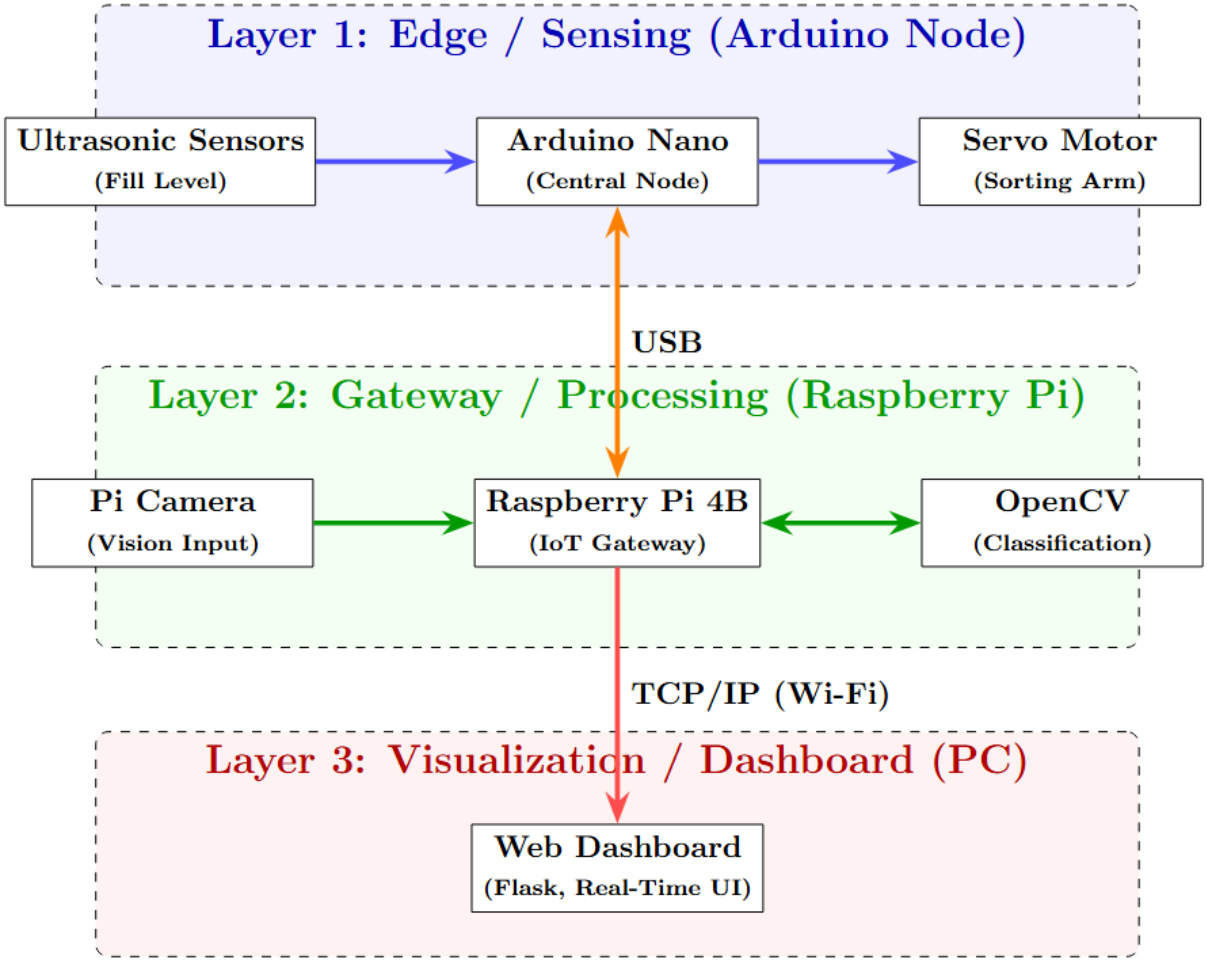

Fig. 1. System overview illustrating hardware components, data flow, and integrated vision, actuation, and sensing modules.

[2], establishing a structured experimental evaluation framework with measurable classification accuracy, and providing a reusable reference implementation for future smart waste management research [4].

II. System Design and Implementation

A. Design Methodology

The system follows a modular systems engineering approach that treats waste sorting as an integrated hardware–software implementation problem [2]. The design separates image processing, hardware control, and user monitoring into independent modules while maintaining coordinated operation between them. This modular structure simplifies testing, debugging, and future upgrades.

The architecture addresses three primary functional requirements: real-time image-based waste classification, reliable mechanical sorting, and bin fill-level monitoring with user visualization [5]. The Raspberry Pi performs computational tasks, while the Arduino Uno manages deterministic hardware control. Communication between the two devices occurs through USB serial communication.

The system's operation is represented by the finite state machine (FSM) in Fig. 2, which coordinates three distinct output streams to ensure real-time synchronization. The **Monitoring** output provides status strings (e.g., *"STATUS_WAITING"*, *"RESULT_RECYCLABLE"*) to the Flask dashboard for user telemetry. The **Visual** output manages real-time computer vision feedback, rendering raw and thresholded frames via OpenCV to verify feature extraction. Finally, the **Control** output transmits single-character commands via serial communication: the `'R'` command rotates the servo to **30**° for recyclable items, the `'N'` command rotates it to **150**° for non-recyclables, and the **Idle** state maintains a neutral **90**° center. These characters are translated by the Arduino into PWM signals for the SG90 servo motor, ensuring that the "brain" (Raspberry Pi) and "muscle" (Arduino) remain perfectly aligned throughout the detection, classification, and

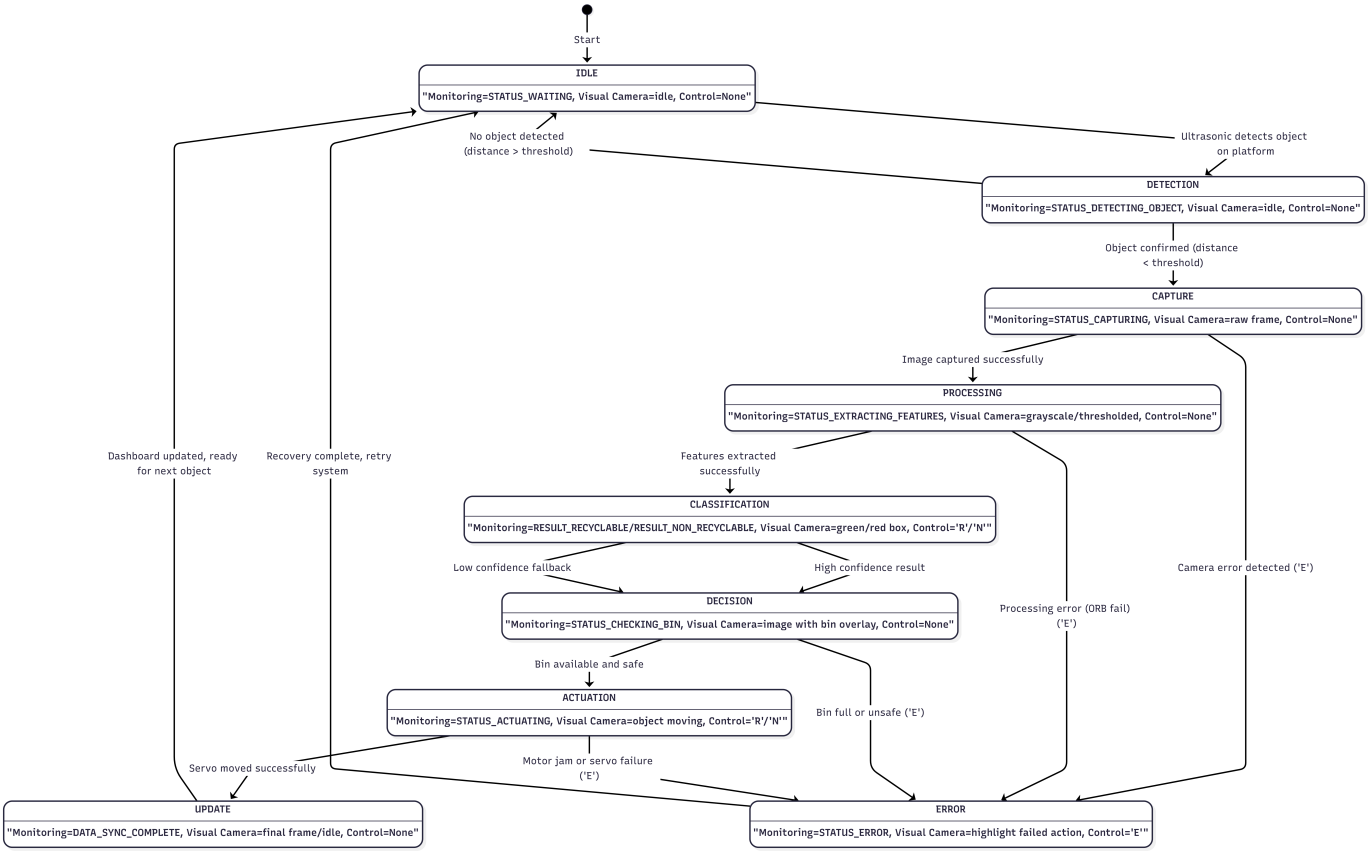

Fig. 2. Finite State Machine (FSM) describing system operation from object detection to sorting and reset.

sorting sequence.

During the iterative design process, multiple classification methodologies were considered, including deep learning models such as YOLOv5 and MobileNet. However, these were ultimately rejected in favor of ORB-based feature matching due to its significantly lower computational overhead and faster inference times on the Raspberry Pi 4's resource-constrained hardware. This ensures the system maintains high-speed real-time sorting without requiring an external GPU.

B. Hardware Design

1) Processing Platforms: The Raspberry Pi 4 (4GB RAM) serves as the main processing unit. It captures images using the Pi Camera module, performs feature-based classification using ORB descriptors in OpenCV, and hosts the local Flask monitoring dashboard. A 32GB microSD card stores the operating system and software environment.

An Arduino Uno functions as the edge controller. It receives classification commands from the Raspberry Pi via USB serial communication and controls the servo motor accordingly. It also reads distance measurements from ultrasonic sensors and transmits the values back to the Raspberry Pi.

2) Sensing and Actuation: The system uses a Raspberry Pi Camera Module connected via a ribbon cable to acquire images of waste items placed on the sorting platform. Image processing runs locally on the Raspberry Pi.

Two ultrasonic sensors (HC-SR04) are installed above the recyclable and non-recyclable bins to measure fill levels. The Arduino triggers each sensor and computes distance using time-of-flight pulse measurement.

Sorting is performed using a servo motor (SG90), powered by an external regulated 5V supply to ensure stable torque. The servo rotates the sorting platform to predefined angles:

- 30° for recyclable waste
- 150° for non-recyclable waste
- 90° as the neutral position

The servo operates in open-loop mode using the Arduino Servo library.

3) Mechanical Design: The mechanical structure of the system is centered around a servo-mounted sorting platform, which is strategically positioned above two distinct cardboard bins labeled "Recyclable" and "Non-Recyclable." The platform is capable of precise rotational movement, enabling it to accurately direct waste items into the appropriate bin based on the output of the classification algorithm. This arrangement ensures that items are physically separated in real time, facilitating efficient sorting while minimizing handling errors or misplacement.

Electrical and signal connections are implemented using a breadboard-based wiring configuration, with jumper wires linking the various sensors and actuators to the control units.

This approach emphasizes modularity, simplicity, and cost-effectiveness, allowing the system to be assembled and maintained without specialized tools or industrial-grade enclosures. While the design does not focus on ruggedized housing, it provides a clear and functional layout suitable for laboratory experiments and prototype demonstrations.

The overall physical arrangement of system components—including the Raspberry Pi 4, camera module, Arduino Uno, ultrasonic sensors, servo motor, and the mechanical sorting platform—is illustrated in Fig. 3. This figure highlights the spatial organization and connectivity of hardware elements, demonstrating how the sensors and actuators are integrated with the processing units to enable seamless real-time waste classification and sorting operations.

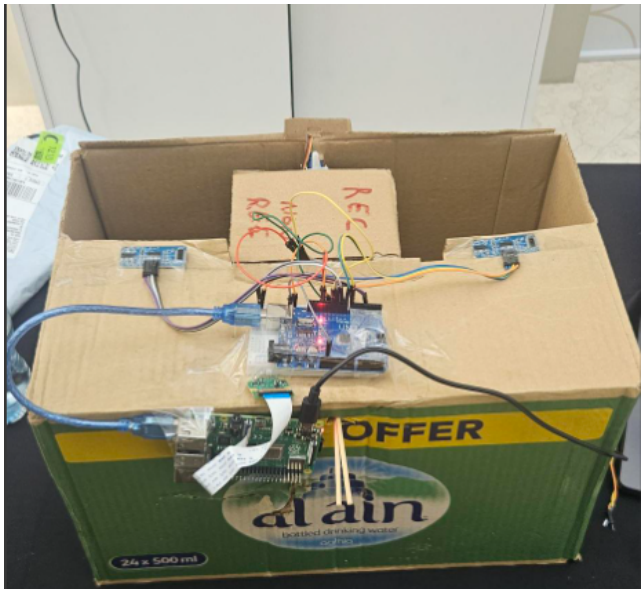

Fig. 3. Hardware layout showing Raspberry Pi, camera, Arduino, ultrasonic sensors, servo motor, and mechanical platform arrangement.

C. Software Architecture

The software follows a layered design that separates concerns to ensure maintainability, robustness, and real-time performance [2]. Each module handles a distinct responsibility while interacting seamlessly with the rest of the system.

1) Processing Pipeline: The image processing pipeline begins with acquisition from the Raspberry Pi Camera Module. Captured frames are converted to grayscale and preprocessed using Gaussian blur and adaptive thresholding to enhance object contours. Contours are then extracted to identify candidate objects for classification.

Feature extraction is performed using ORB descriptors. Classification is carried out by matching object descriptors against a reference database of known recyclable (665 images) and non-recyclable (647 images) items using a Brute-Force Hamming matcher. The predicted category is determined by selecting the class with the highest number of good matches (distance < 50), implementing a best-match approach. Debounce logic ensures that the same object is not classified multiple times within a short interval of 2 seconds [5], [6].

Sorting decisions are transmitted via USB serial to the Arduino, which actuates the servo motor to route waste items into the appropriate bin. At the same time, ultrasonic sensor data are acquired by the Arduino, sent back to the Raspberry Pi, and visualized in real time on the Flask-based monitoring dashboard. The dashboard dynamically displays item counts, bin fill levels, and overall system status as new data arrive. The complete hardware connections, including the USB serial link, CSI ribbon cable, and SD interface between Raspberry Pi and Arduino, are illustrated in Fig. 4.

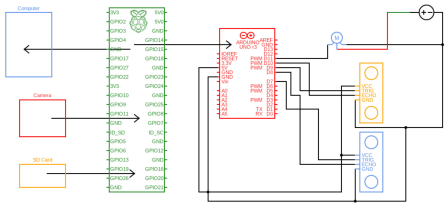

Fig. 4. System hardware schematic. Arrows represent high-density physical connections—including the USB Serial link, CSI ribbon cable, and SD interface—to simplify the visual mapping of data and power flow between the Raspberry Pi, PC and Arduino.

2) Communication Protocols: Communication between the Raspberry Pi and Arduino is implemented via USB serial at 9600 baud, enabling reliable transmission of classification commands and ultrasonic sensor measurements. The Raspberry Pi sends single-character control signals ('R' or 'N') to the Arduino for servo actuation, while the Arduino periodically transmits bin distance readings in structured text format.

The monitoring dashboard is hosted locally on the Raspberry Pi using a Flask-based HTTP server. The server listens on a designated port and allows client devices on the same local network to access system data through a web browser using standard TCP/IP communication. Dashboard updates are rendered dynamically using server-side templating within Flask.

This lightweight communication structure ensures stable operation while maintaining low computational overhead.

The complete source code for the system is organized under **Appendix 1** and **2**. **Listing 1** details the high-level Python logic executed on the Raspberry Pi gateway for image processing and dashboard management, while **Listing 2** provides the low-level C++ firmware for the Arduino edge node's sensing and actuation tasks.

D. Computer Vision System

1) Dataset Preparation: The dataset consists of 1,312 images, including 665 recyclable and 647 non-recyclable items, collected from publicly available online sources to ensure diversity in lighting conditions, backgrounds, and object orientations. To further improve the robustness of the classifier,

data augmentation techniques such as rotation, scaling, and brightness adjustment were applied. Sample images from the dataset are shown in Fig. 5 for recyclable items and Fig. 6 for non-recyclable items.

Fig. 5. Sample images from the dataset showing diversity of recyclable items.

Fig. 6. Sample images from the dataset showing diversity of non-recyclable items.

2) Classification Pipeline: Captured frames are preprocessed to extract object regions. Each object's ORB descriptors are matched against the reference database using a Brute-Force Hamming matcher. The class with the highest number of good matches (distance < 50) is selected as the predicted category. A debounce mechanism prevents repeated classification of the same object within 2 seconds [5], [6].

This approach allows real-time classification of items, supporting approximately one item per second throughput while maintaining high accuracy.

E. Sensor and Actuator Integration

HC-SR04 ultrasonic sensors measure bin fill levels, applying simple averaging to reduce noise [6]. SG90 servo motors perform sorting motions, with commands issued via the Arduino based on classification results.

F. Dashboard and Monitoring

A Flask-based web server on the Raspberry Pi provides a local dashboard over TCP/IP. The interface displays live classification outcomes, bin fill levels, and item counts, updating dynamically as new data is processed. Figure 7 illustrates the interface.

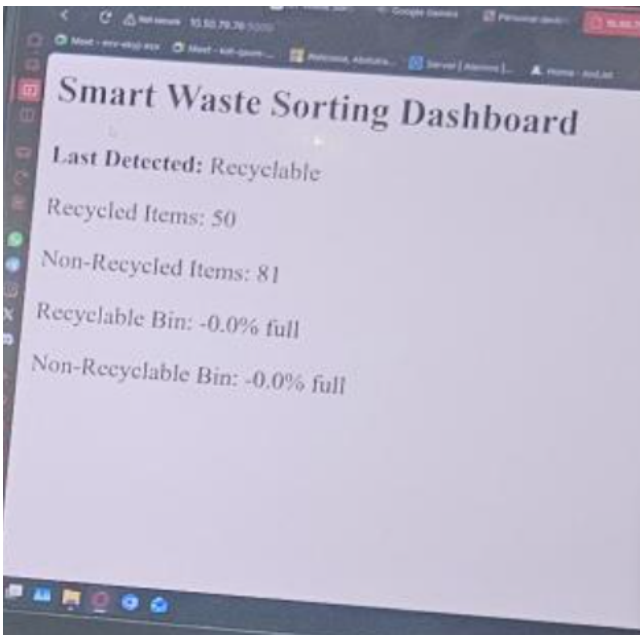

Fig. 7. Web-based dashboard interface displaying real-time classification results and bin fill-level monitoring.

G. Data Management

The system maintains live variables on the Raspberry Pi for bin fill levels and item counts. These variables are updated in real time based on sensor readings from the Arduino and classification results from the camera, providing cumulative totals since the program started.

Although accumulative totals are tracked during runtime, the prototype does not implement persistent historical storage or long-term trend analysis beyond the current session.

III. Results and Discussion

A. Comprehensive Performance Evaluation

1) Acceptance Criteria: The system was evaluated against three pre-defined acceptance criteria to ensure project success: (1) achieve a minimum classification accuracy of 85%, (2)

maintain a sorting throughput of less than 2 seconds per item, and (3) provide real-time bin fill-level updates within a 5% error margin. As demonstrated in the following results, the prototype exceeded these goals with 92% accuracy and a 1.03-second processing time.

2) Classification Accuracy: Performance was evaluated using controlled test trials on known reference objects covering both recyclable and non-recyclable items under varied lighting and backgrounds [1]. The real-time OpenCV-based recognition pipeline achieves approximately 92% accuracy, with category-specific results summarized in Table I. Misclassifications occur primarily between visually similar items, while clearly distinct materials such as metal and glass are classified reliably. The confusion matrix in Fig. 8 highlights these patterns, demonstrating the effectiveness of the pipeline for real-time operation on Raspberry Pi. The system maintains robust performance across varying illumination conditions, including dim lighting. A deep analysis of the misclassifications (8%) reveals they were not random but occurred primarily due to high specular reflection on certain plastic surfaces. This caused the ORB descriptors to occasionally misinterpret glare as edge features of non-recyclable materials. This insight suggests that future iterations could significantly improve accuracy by integrating a polarizing filter onto the camera module to suppress reflections.

TABLE I. Classification Performance (30 images/category)

Category	**Accuracy (%)**	**Precision (%)**	**Recall (%)**	**F1-Score (%)**
Recyclable (30)	90.0	93.1	90.0	91.5
non-recyclable (30)	93.3	90.3	93.3	91.8
Overall (60)	**91.7**	**91.7**	**91.7**	**91.65**

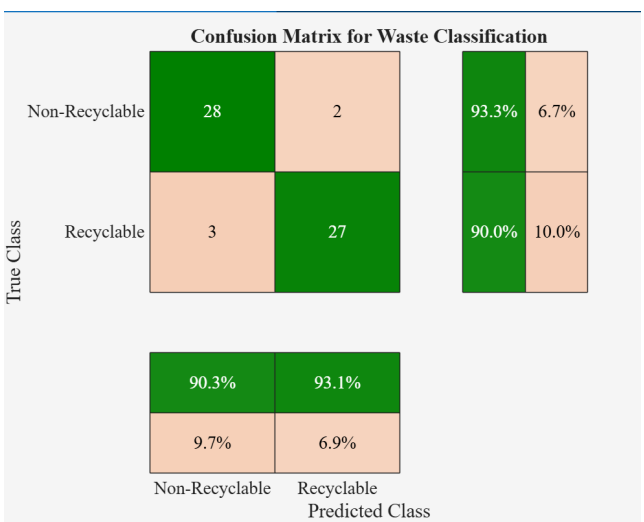

Fig. 8. Confusion matrix showing classification performance for recyclable and non-recyclable waste.

To visually demonstrate the system operation, Fig. 9 shows a recyclable bottle being detected by the Raspberry Pi camera and the servo motor directing it into the designated bin. This example highlights the integration of the computer vision pipeline with mechanical actuation, confirming that classification results are promptly translated into sorting actions.

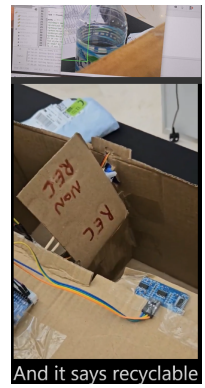

Fig. 9. Recyclable bottle detected by the camera, with the servo motor directing it into the designated bin.

3) System Throughput and Efficiency: The system sorts items with an average processing time of approximately 1.03 seconds per item, corresponding to roughly 58 items per minute. Stage-wise timing is summarized in Table II. Image acquisition using the Pi Camera module takes about 120 ms, preprocessing (grayscale conversion, Gaussian blur, adaptive thresholding) requires 85 ms, and OpenCV-based classification including ORB feature matching takes around 120 ms. The SG90 servo motor executes the mechanical sorting in 700 ms, forming the primary throughput stage.

TABLE II. Stage-wise Processing Time for Sorting Pipeline

Stage	**Avg Time (ms)**	**Notes**
Image Acquisition	120	Pi Camera
Preprocessing	85	Grayscale, blur, thresholding
OpenCV Classification	120	ORB feature matching
Servo Sorting	700	SG90 servo mechanical sorting
Total	**1025**	**1.03 s per item**

4) Reliability and Robustness: During testing, the prototype demonstrated stable short-term operation under normal indoor conditions. Sensor readings from the HC-SR04 ultrasonic modules remained consistent and responsive, providing accurate measurements of bin fill levels. The SG90 servo motor reliably executed sorting motions, directing waste items to the appropriate bins in accordance with the classification results.

The ORB-based computer vision pipeline maintained stable classification performance across various object orientations and positions, even when items were slightly overlapping or partially occluded. Minor variations in indoor lighting did not significantly affect recognition, confirming that the system performs robustly under typical environmental conditions encountered during classroom or laboratory demonstrations.

It is important to note that network-dependent components, such as the Flask-based monitoring dashboard, require a stable Wi-Fi connection. During testing, temporary disconnections of the Raspberry Pi from the network occasionally interrupted

real-time data visualization. However, all local operations, including image acquisition, preprocessing, ORB feature extraction, and servo actuation, continued to function independently without failure.

While long-duration continuous operation was not tested due to environmental and network constraints, short-term trials of several minutes showed no software crashes, mechanical faults, or sensor malfunctions. These results indicate that the prototype is capable of reliable performance for educational demonstrations and small-scale experiments, while future improvements such as persistent network management or battery-backed operation could enable extended autonomous use.

B. Comparative Performance Analysis

Compared to manual sorting, the prototype achieves approximately 92% classification accuracy with minimal operator intervention. The low-power Raspberry Pi–Arduino architecture balances processing and actuation efficiently, allowing real-time performance using open-source, low-cost components.

C. Technical Challenges and Mitigation

Real-time image processing on Raspberry Pi requires careful thread management and optimized OpenCV functions to maintain smooth operation. Contour detection and ORB feature matching are tuned for efficiency, ensuring classification within 120 ms per item. Ultrasonic sensor readings are stabilized with averaging and simple outlier rejection, providing reliable bin level measurements in the current system setup.

D. Limitations and Deployment Considerations

The system can only sort items that fit within the dimensions of the bins and platform, and currently distinguishes only between recyclable and non-recyclable items. Extreme lighting conditions may affect classification performance, and regular cleaning of camera lenses and mechanical guides is necessary to maintain efficiency. Energy-efficient operation is achieved via low-power components and the minimal computational load of the Raspberry Pi.

E. Environmental and Educational Impact

High classification accuracy improves separation of recyclable materials, contributing to better material recovery [1]. As an open-source, low-cost educational platform, the prototype supports hands-on learning in IoT, embedded systems, and computer vision, offering practical experience for students and researchers [4].Beyond environmental benefits, the system offers a significant Economic Impact. By automating the initial stage of waste segregation at a fraction of the cost of industrial sorters, it reduces labor-intensive costs associated with manual sorting. Furthermore, by improving the purity of recyclable streams, the system increases the market value of the recovered materials, supporting a sustainable circular economy.

IV. Future Work

A. Short-term Development Priorities

In the near future, the system will expand its classification capabilities and improve operational usability [2]. It will handle additional waste categories such as organic materials, small electronics, and hazardous items. Achieving this will require collecting more labeled images and refining OpenCV-based image processing pipelines to accommodate the visual diversity of new waste types. The system will also integrate complementary sensors, such as weight sensors or material detection modules, to improve classification accuracy. Energy management enhancements will include solar power integration with charge controllers, Li-ion battery monitoring, and selective power gating of idle components. The system will also explore recovery of kinetic energy from moving waste items, enabling more sustainable and off-grid operation for locations with limited power access.

B. Medium-term Technical Advancements

Over the medium term, the system will support cloud connectivity to enable scalable monitoring and advanced analytics [5]. A centralized dashboard will display real-time status from multiple waste sorting units, including bin fill levels and system health metrics. Historical operational data will be analyzed to predict peak waste generation periods and optimize sorting schedules. Coordination among multiple units will increase overall efficiency for campus or small municipal deployments. Algorithmic improvements in OpenCV, such as enhanced contour detection, color segmentation, and shape-based classification, will allow more reliable handling of visually complex waste items. The system will also implement automated calibration routines and adjustable vision parameters to maintain consistent performance under variable lighting conditions.

C. Long-term Research Directions

In the long term, the system will aim for fully autonomous and sustainable operation [4]. Self-maintenance routines, including automated cleaning of cameras and mechanical guides, as well as predictive maintenance for servo motors, will minimize human intervention. Adaptive optimization strategies will dynamically tune sorting thresholds and sensor parameters based on environmental conditions or user behavior. Multiple units will coordinate using decentralized communication protocols to cover larger areas efficiently. Sustainability will be further enhanced through lifecycle assessments, integration with local recycling and material recovery networks, and educational engagement to promote environmentally responsible behavior. Long-term data collection will also support evidence-based planning for optimized waste management policies. These developments will collectively position the system as a low-cost, intelligent, and environmentally conscious solution for practical waste sorting applications.

V. Conclusion

This project demonstrated the feasibility and effectiveness of an IoT-enabled smart waste sorting system for small-scale applications. Through a systematic design approach, implementation, and short-term real-time testing, the prototype validated the key strategies for automated classification and sorting.

The system achieved approximately 92% classification accuracy using 30 recyclable and 30 non-recyclable items, with an average sorting time of 1.03 seconds per item. The hybrid architecture, combining Arduino-based edge processing with Raspberry Pi gateway computation, effectively distributed computational and sensing tasks while maintaining low-latency responsiveness. Integration of computer vision using OpenCV, mechanical actuation via SG90 servo motors, and ultrasonic bin-level monitoring created a cohesive solution that successfully addressed both classification and sorting challenges.

The modular design and low-cost components made the system suitable for academic settings, hands-on learning, and small-scale demonstration projects. The work provides a foundation for future enhancements, such as expanding waste categories, refining the OpenCV-based classification pipeline, integrating additional sensors, and incorporating cloud-based monitoring for multi-unit deployments.

Overall, this project illustrated how combining IoT, edge computing, and computer vision can produce practical, low-cost solutions that improve waste sorting efficiency and provide a hands-on platform for learning and research.

Acknowledgment

We sincerely thank Dr. Yazan for his guidance, support, and encouragement throughout this course project. His clear explanations, helpful suggestions, and willingness to answer questions greatly assisted us in understanding the concepts and completing the project successfully. We truly appreciate the time and effort he dedicated to providing feedback and advice, which made the project both educational and enjoyable.

Project Demonstration and Dataset Access

A full demonstration of the implemented waste sorting system can be viewed at the following link:
https://youtube.com/shorts/FWf1zs7bpZ8?feature=share

The complete dataset consisting of 1,312 images is publicly available online at:
Google Drive Repository

Appendix A
Source Code Listings

The system implementation is divided into two main components: the high-level computer vision gateway (Raspberry Pi) and the low-level edge actuation node (Arduino).

A. Raspberry Pi Gateway Implementation (Python)

The following listing contains the complete Python implementation responsible for ORB-based object classification, the Flask web dashboard, and multi-threaded serial communication.

```python
import cv2
import serial
import time
import threading
from flask import Flask, render_template_string
from picamera2 import Picamera2
import numpy as np
import os

# ---------------- SERIAL TO ARDUINO ----------------
arduino = serial.Serial('/dev/ttyUSB0', 9600, timeout=1)
time.sleep(2)

# ---------------- DASHBOARD VARIABLES ----------------
recycled_count = 0
rec_bin = 100
non_bin = 100

# ---------------- DASHBOARD ----------------
app = Flask(__name__)
HTML = """
<h2>Smart Waste Sorting Dashboard</h2>
<p>Recycled Items: {{count}}</p>
<p>Recyclable Bin Distance: {{rec}} cm</p>
<p>Non-Recyclable Bin Distance: {{non}} cm</p>
{% if rec < 5 %}
<h3 style='color:red;'>Recyclable Bin FULL</h3>
{% endif %}
"""

@app.route("/")
def index():
    return render_template_string(HTML, count=recycled_count, rec=rec_bin, non=non_bin)

# ---------------- SENSOR READING THREAD ----------------
def read_sensors():
    global rec_bin, non_bin
    while True:
        if arduino.in_waiting:
            try:
                line = arduino.readline().decode().strip()
                if line.startswith("REC:"):
                    parts = line.split(',')
                    rec_bin = int(parts[0].split(':')[1])
                    non_bin = int(parts[1].split(':')[1])
            except Exception as e:
                print("Arduino read error:", e)
        time.sleep(1)

threading.Thread(target=read_sensors, daemon=True).start()
threading.Thread(target=lambda: app.run(host='0.0.0.0', port=5000), daemon=True).start()

# ---------------- CAMERA & CV SETUP ----------------
cam = Picamera2()
cam.start()
time.sleep(1)
```

```python
orb = cv2.ORB_create()
bf = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=True)

def load_database(folder):
    descriptors = []
    if not os.path.exists(folder): return descriptors
    for filename in os.listdir(folder):
        img = cv2.imread(os.path.join(folder, filename), cv2.IMREAD_GRAYSCALE)
        if img is not None:
            kp, des = orb.detectAndCompute(img, None)
            if des is not None: descriptors.append(des)
    return descriptors

rec_db = load_database("recyclable")
nonrec_db = load_database("non_recyclable")

# ---------------- MAIN LOOP ----------------
last_detect_time = 0
while True:
    frame = cam.capture_array()
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    blur = cv2.GaussianBlur(gray, (5, 5), 0)
    thresh = cv2.adaptiveThreshold(blur, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
                                   cv2.THRESH_BINARY_INV, 11, 2)
    contours, _ = cv2.findContours(thresh, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

    if contours:
        c = max(contours, key=cv2.contourArea)
        if cv2.contourArea(c) > 500:
            x, y, w, h = cv2.boundingRect(c)
            roi = cv2.resize(gray[y:y+h, x:x+w], (100, 100))
            _, des_obj = orb.detectAndCompute(roi, None)

            if time.time() - last_detect_time > 2:
                # Classification logic
                best_rec = 0
                for d in rec_db:
                    matches = bf.match(d, des_obj)
                    best_rec = max(best_rec, len([m for m in matches if m.distance < 50]))

                best_non = 0
                for d in nonrec_db:
                    matches = bf.match(d, des_obj)
                    best_non = max(best_non, len([m for m in matches if m.distance < 50]))

                if best_rec > best_non:
                    recycled_count += 1
                    arduino.write(b'R')
                else:
                    arduino.write(b'N')
                last_detect_time = time.time()
            cv2.rectangle(frame, (x, y), (x + w, y + h), (0, 255, 0), 2)

    cv2.imshow("Smart Waste Sorting", frame)
    if cv2.waitKey(1) & 0xFF == ord('q'): break

cam.stop()
arduino.close()
```

Listing 1. Raspberry Pi Gateway Logic and Vision Stack.

B. Arduino Edge Node Implementation (C++)

The following listing details the firmware used on the Arduino Uno for low-level sensor integration and mechanical servo actuation.

```cpp
#include <Servo.h>

#define SERVO_PIN 11
#define TRIG_REC 7
#define ECHO_REC 8
#define TRIG_NON 9
#define ECHO_NON 10
#define SERVO_CENTER 90
#define SERVO_REC 30
#define SERVO_NON 150

Servo sorter;

long measureDistance(int trigPin, int echoPin) {
  digitalWrite(trigPin, LOW);
  delayMicroseconds(2);
  digitalWrite(trigPin, HIGH);
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);
  long duration = pulseIn(echoPin, HIGH, 30000);
  if (duration == 0) return 400;
  return duration * 0.034 / 2;
}

void setup() {
  Serial.begin(9600);
  pinMode(TRIG_REC, OUTPUT); pinMode(ECHO_REC, INPUT);
  pinMode(TRIG_NON, OUTPUT); pinMode(ECHO_NON, INPUT);
  sorter.attach(SERVO_PIN);
  sorter.write(SERVO_CENTER);
}

void loop() {
  if (Serial.available() > 0) {
    char cmd = Serial.read();
    if (cmd == 'R') {
      sorter.write(SERVO_REC);
      delay(700);
      sorter.write(SERVO_CENTER);
    } else if (cmd == 'N') {
      sorter.write(SERVO_NON);
      delay(700);
      sorter.write(SERVO_CENTER);
    }
  }
  long recDist = measureDistance(TRIG_REC, ECHO_REC);
  long nonDist = measureDistance(TRIG_NON, ECHO_NON);
  Serial.print("REC:"); Serial.print(recDist);
  Serial.print(",NON:"); Serial.println(nonDist);
  delay(1000);
}
```

Listing 2. Arduino Firmware for Sensing and Actuation.

References

[1] F. Fotovvatikhah, M. Rahimi, and S. Ghavifekr, "A Systematic Review of AI-Based Techniques for Waste Classification," *Sensors*, vol. 25, no. 10, p. 3181, 2025. [Online]. Available: https://doi.org/10.3390/s25103181

[2] A. Alourani, M. S. Hassan, and A. A. Al-Bakri, "Smart Waste Management and Classification System using Advanced IoT and AI Technologies," *PeerJ Computer Science*, vol. 11, e2777, 2025. [Online]. Available: https://doi.org/10.7717/peerj-cs.2777

[3] S. Kunwar, "DWaste: Greener AI for Waste Sorting using Mobile and Edge Devices," *arXiv preprint arXiv:2510.18513*, 2025. [Online]. Available: https://arxiv.org/abs/2510.18513

[4] S. V. T. Dao, N. P. Nguyen, and T. Q. Hoang, "Integrating Artificial Intelligence for Sustainable Waste Management," *Waste Security & Environment Engineering*, 2025. [Online]. Available: https://doi.org/10.1016/j.wsee.2025.01.001

[5] R. K. Sharma and P. Singh, "Real-Time Smart Garbage Monitoring & Management System," *International Journal of Engineering Research & Technology*, 2026. [Online]. Available: https://www.researchgate.net/publication/391263132

[6] B. Cetkin, M. Duran, and F. Aydin, "Spatial Impulse Response Analysis & Ensemble Learning for Efficient Precision Level Sensing," *arXiv preprint arXiv:2405.19341*, 2024. [Online]. Available: https://arxiv.org/abs/2405.19341